Adaptive Control of a DC Motor

Lenard Wild, Felix Raffl

1 Introduction

Example 7.6 of the reader cites the experiment by Abram and Fieg, where the adaptive controller is implemented on an Arduino Due [1, 2]. The present section realises the same self-tuning pole-placement idea on an own test rig. The hardware consists of an ESP32, an L298N H-bridge, and a DC motor with a quadrature encoder. The controller is not hand-coded on the microcontroller; instead, the real-time code is generated automatically from the Simulink model.

The structure follows the indirect adaptive-control scheme of Figure 7.1 in the reader: the plant parameters are estimated recursively, the controller is redesigned from these estimates, and the resulting incremental PI algorithm is executed on the ESP32 input/output interface. Figure 1 shows the arrangement for the motor rig. The first-order controller runs on the hardware, while the second-order extension is studied in simulation.

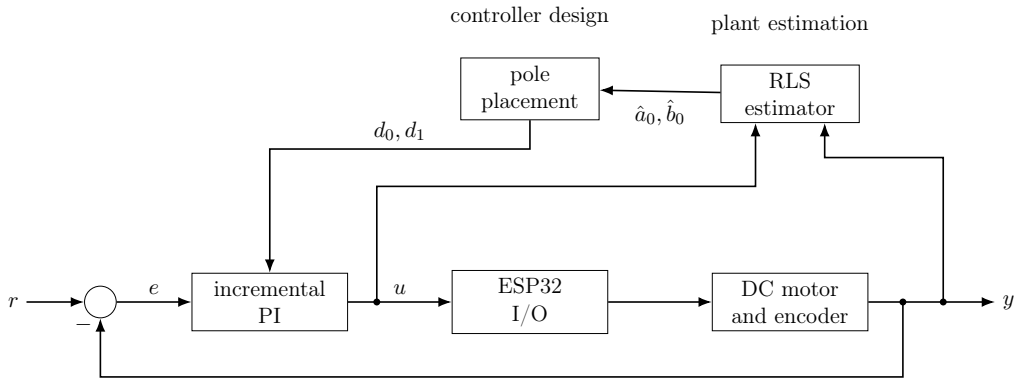

Figure 1: Self-tuning regulator used for the ESP32 DC motor rig.

2 First-order design

This mirrors the derivation of Example 7.6 of the reader. The first-order armature-to-speed relation is

$$J\dot{\omega} = -b\omega + K_m u. \tag{1}$$

Its continuous-time transfer function is

$$G(s) = \frac{\omega(s)}{u(s)} = \frac{K_m/J}{s + b/J}. \tag{2}$$

With zero-order hold sampling and $T = 0.08\,\mathrm{s}$, the discrete transfer function is written as

$$G(z) = \frac{b_0}{z + a_0}, \tag{3}$$

where

$$a_0 = -\exp\!\left(-\frac{bT}{J}\right), \qquad b_0 = \frac{K_m}{b}\left(1 - \exp\!\left(-\frac{bT}{J}\right)\right). \tag{4}$$

The identified parameter vector is therefore

$$x_e = \begin{bmatrix} a_0 & b_0 \end{bmatrix}^T. \tag{5}$$

The discrete plant model in (3) gives the one-step regression

$$y(k) = C(k)x_e(k), \qquad C(k) = \begin{bmatrix} -y(k-1) & u(k-1) \end{bmatrix}. \tag{6}$$

The recursive least-squares update with forgetting factor α is

$$L(k) = \frac{P(k-1)C^T(k)}{C(k)P(k-1)C^T(k) + \alpha}, \tag{7}$$

$$x_e(k) = x_e(k-1) + L(k)\left(y(k) - C(k)x_e(k-1)\right), \tag{8}$$

$$P(k) = \frac{\left(I - L(k)C(k)\right)P(k-1)}{\alpha}. \tag{9}$$

The initial values are

$$P(0) = 100I, \qquad x_e(0) = \begin{bmatrix} -0.5 & 1.0 \end{bmatrix}^T. \tag{10}$$

The covariance update is suspended without excitation, which prevents covariance windup. The controller-design step uses a guard $|b_0| > 10^{-3}$, avoiding division by a nearly singular plant gain.

The desired closed-loop polynomial is chosen as

$$q(z) = z^2 - 0.5z + 0.06 = (z - 0.3)(z - 0.2). \tag{11}$$

For $T = 0.08\,\mathrm{s}$, these discrete poles correspond to continuous poles close to $-15\,\mathrm{s}^{-1}$ and $-20\,\mathrm{s}^{-1}$. The target settling time is therefore about 250 ms. Writing $q(z) = z^2 + q_1 z + q_0$, the controller parameters are computed from the current plant estimates by

$$d_0 = \frac{q_0 + a_0}{b_0}, \qquad d_1 = \frac{q_1 + 1 - a_0}{b_0}. \tag{12}$$

The implemented controller is the incremental PI law

$$u(k) = u(k-1) + d_1 e(k) + d_0 e(k-1), \tag{13}$$

where $e(k) = r(k) - y(k)$. The saturated command, not the unsaturated internal value, is fed back as $u(k-1)$ in (13); this is the anti-windup mechanism used in the implementation.

The transient cannot be shaped by the pole locations alone, because the closed-loop transfer function contains the controller zero

$$z_0 = -\frac{d_0}{d_1}. \tag{14}$$

This is the same structural effect described in the reader as a phase error caused by the different numerator order. A one-degree-of-freedom controller provides no independent numerator design and cannot remove the zero without changing the controller structure.

3 Second-order design

With armature inductance L_a, armature resistance R_a, inertia J, viscous friction b, and motor constant K_m, the continuous-time second-order transfer function is

$$G(s) = \frac{K_m}{L_a J s^2 + (L_a b + R_a J)\, s + (R_a b + K_m^2)}. \tag{15}$$

After zero-order-hold discretisation this is written as

$$G(z) = \frac{b_1 z + b_0}{z^2 + a_1 z + a_0}. \tag{16}$$

The adaptive estimator therefore has four parameters, with regression vector

$$\varphi(k) = \begin{bmatrix} -y(k-1) & -y(k-2) & u(k-1) & u(k-2) \end{bmatrix}. \tag{17}$$

Example 7.6 of the reader does not use the free $c(z)$ of the general Diophantine system; it forces $c(z) = z - 1$, i.e. a PI controller. Our second-order design does the same thing one order

higher. Pure pole placement fixes only the poles, not the static gain, and therefore leaves a steady-state error; the simulated value was 7.8 percent. To remove this error, the design forces

$$c(z) = (z-1)(z+c_0), \tag{18}$$

which gives $T(1) = 1$. Equation 7.16 of the reader gives the general design equation whose linear system is solved for the controller coefficients at every sampling instant.

The decisive point is the sampling time. For

$$K_m = 0.05, \quad J = 0.001\,\mathrm{kg\,m^2}, \quad b = 0.0005, \quad L_a = 2\,\mathrm{mH}, \quad R_a = 2\,\Omega,$$

the continuous poles are $-998.7\,\mathrm{s}^{-1}$ for the electrical mode and $-1.75\,\mathrm{s}^{-1}$ for the mechanical mode. The electrical time constant is therefore about 1 ms. Table 1 shows the exact ZOH discretisation at the hardware sampling time and at a sampling time that resolves the electrical pole.

Table 1: Exact ZOH parameters of the second-order motor model.

T	a_1	a_0	b_1	b_0	z-poles	
80 ms	−0.8692	0.000000	1.8467	0.0218	0.8692	0
2 ms	−1.1322	0.1352	0.0284	0.0148	0.9965	0.1357

At $T = 80\,\mathrm{ms}$, the electrical pole is mapped to $z = 0$ and has fully decayed between two samples. Two of the four parameters then no longer contribute independent information, the regressor loses rank, and the plant is structurally not identifiable. The closed loop can nevertheless still follow the reference stably, even though the parameter estimate cannot converge under that sampling, because the controller only needs certain combinations of the parameters, not their individual values. A stable closed loop is therefore not evidence of successful identification. At $T = 2\,\mathrm{ms}$, the poles are 0.9965 and 0.1357, and all four parameters are identifiable.

The working design uses $T = 2\,\mathrm{ms}$, with PRBS dither added to the manipulated variable. Clipping does not corrupt the estimate because the regressor uses the post-saturation command. The estimator is used without forgetting, $\alpha = 1$, because without persistent excitation $\alpha < 1$ makes P grow without bound.

The desired closed-loop polynomial is

$$q(z) = z(z-0.1357)(z-0.9608)^2. \tag{19}$$

The fast electrical pole is deliberately left in place in (19) because it is already much faster than the desired bandwidth. If instead

$$q(z) = (z-0.9608)^4$$

is demanded, the design slows the electrical pole by a factor of 50. The controller must then invert the fast plant dynamics and becomes unstable itself, with its pole at $|z| = 2.66$, and behind a saturation limit such a controller necessarily diverges. The design rule is therefore to leave modes that are much faster than the desired bandwidth where they are.

4 Simulation results

The validation run uses the second-order design with $T = 2\,\mathrm{ms}$. Table 2 shows that all four parameters converge to their true ZOH values. In the window $t = 3\,\mathrm{s}$ to $4\,\mathrm{s}$, all four parameters lie within 0.65 % relative error of their true ZOH values.

The static closed-loop gain is $T(1) = 1.000$. For the step at $t = 5.5\,\mathrm{s}$, the response has no overshoot and the 2% settling time is 0.384 s. The trace of P remains bounded at 400. Figure 2 shows the output, command, parameter estimates, and covariance trace from this validation run.

Table 2: Validated second-order parameter estimates at $T = 2\,\mathrm{ms}$.

Parameter	Estimated, $t = 3\,\mathrm{s}$ to $4\,\mathrm{s}$	True	Absolute error
a_1	-1.131310	-1.132176	8.66×10^{-4}
a_0	0.134337	0.135200	8.63×10^{-4}
b_1	0.028348	0.028362	1.49×10^{-5}
b_0	0.014889	0.014832	5.71×10^{-5}

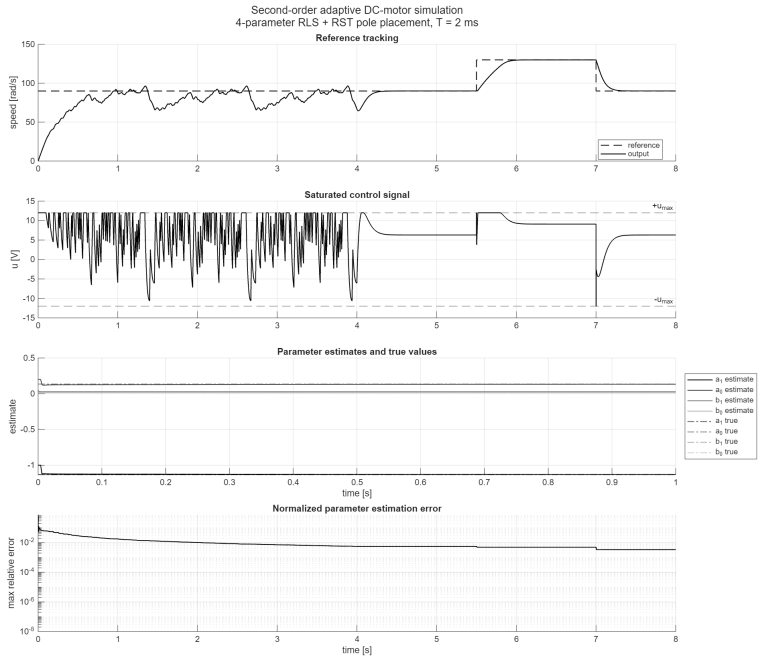

Figure 2: Validation run of the second-order adaptive RST controller.

5 Hardware realisation

The hardware platform is an ESP32-WROOM module connected to an L298N H-bridge. The L298N introduces an approximate voltage drop of $4\,\mathrm{V}$, so with a $12\,\mathrm{V}$ supply the motor sees about $8\,\mathrm{V}$. The motor is equipped with a quadrature encoder. The sampled controller uses the period $T = 0.08\,\mathrm{s}$, $P(0) = 100I$, $x_e(0) = \begin{bmatrix} -0.5 & 1.0 \end{bmatrix}^T$, forgetting factor $\alpha = 0.98$, desired polynomial $q(z) = z^2 - 0.5z + 0.06$, and a command saturated at ± 255.

The controller is implemented by automatic code generation from Simulink using the Embedded Real-Time (ERT) target, the ESP32 target support, and External Mode for Monitor and Tune operation. Table 3 gives the pin assignment.

Table 3: ESP32 pin assignment for the DC motor test rig.

Signal	ESP32 pin
PWM	GPIO 14
IN1	GPIO 26
IN2	GPIO 27
Encoder A	GPIO 32
Encoder B	GPIO 33

The encoder resolution was determined by a one-revolution test as 44 counts per revolution, corresponding to 11 encoder impulses with x4 quadrature edge decoding. The speed conversion

$$\omega = \frac{2\pi \Delta N}{T\,\mathrm{CPR}}$$

is therefore calibrated for CPR = 44. The measured full-duty speed of approximately 780 rad/s corresponds to about 7450 rpm, consistent with the gearbox-free 25GA371 speed range of roughly 3000 rpm to 8000 rpm and independently supporting the encoder scaling.

6 Conclusion

The first-order self-tuning regulator from the reader runs on the ESP32 rig with the encoder scaling fixed by the one-revolution test at 44 counts per revolution. The same derivation carried to the second-order plant identifies all four parameters in simulation at $T = 2\,\text{ms}$. At the hardware sampling time of $T = 80\,\text{ms}$, the second-order plant is structurally not identifiable, which is why the second-order design stays a simulation result.

References

[1] A. Ravazzolo-Mehrle, *Advanced Control Engineering 2 - Optimal Control.* MCI Innsbruck, 2026. Reader, spring term 2026.

[2] Abram and Fieg, "Adaptive control experiment with an arduino due." cited in [Reader].

[3] K. J. Aström and B. Wittenmark, *Adaptive Control.* Addison-Wesley, 2 ed., 1995.

[4] L. Ljung, *System Identification - Theory for the User.* Prentice Hall, 2 ed., 1999.